

object-oriented programming(OOP)

An Object oriented Programming is a pattern to write programming language in the form of classes and objects so that we can relate it to real world.

Classes

A class is a design or blueprint that defines data and behaviour of object to be created.

```
class Dog {  
    // Fields (properties)  
    String name;  
    int age;  
    String breed;  
  
    // Method (behavior)  
    void bark() {  
        System.out.println(name + " is barking!");  
    }  
  
    void eat() {  
        System.out.println(name + " is eating.");  
    }  
}
```

Objects

An Object is a real world entity which has a physical form which has some properties and behaviour. similarly in JAVA, An Object is a instance of class created to access its data and operations.

```
public class Main {  
    public static void main(String[] args) {  
  
        Dog dog1 = new Dog();    // Object creation  
        dog1.name = "Buddy";  
        dog1.age = 3;  
        dog1.breed = "Golden Retriever";  
  
        dog1.bark();  
        dog1.eat();  
    }  
}
```

Using `new` keyword we can create Object

```
new Dog();
```

Then we can store this object created/instantiated into a reference variable named dog1

```
Dog dog1 = new Dog();
```

Here, Dog is a variable of Datatype(Non primitive) Dog, same int is a Datatype (primitive).

dog1 is a reference variable as it stores reference/address in the method stack of the instantiated object which is in heap memory of JVM.

Let us understand what is happening,

1. When we write java code in some file with extension .java
2. It is just a file stored in our internal/external memory
3. When we compile this using `javac Dog.java` this created a bytecode i.e. Dog.class

4. This is also stored in our internal/external memory
5. But this a blueprint/design of Dog class that the java compiler has converted into a format that JVM knows how to create it.
6. Once we run interpreter JRE using class name ,using `java Dog`
7. The Interpreter will instantiate JVM which will occupy some space on RAM
8. Once JVM is instantiated from main method, it will start interpreting code one by one written in main method.
9. Once it will come to the line `Dog dog1 = new Dog();`
10. It will create a Dog object in Heap memory of JVM which is a dynamic memory which can expand based on memory requirements.
11. JVM will store dog1 into the main stack with the reference of the Dog object created in heap memory.
12. Using this reference we can access the properties(fields/class variables) and behaviours(class methods).

Output:

```
csharp
```

```
Buddy is barking!
```

```
Buddy is eating.
```

You can check the reference value or the address by printing the reference variable directly

```
System.out.println(dog1); // printing reference variable
```

Output (something like):

```
CSS
```

```
Dog@6d06d69c
```

This is hashcode from the Object class will see in further lectures.

